Secure Challenge Derivation in ZisK

Héctor Masip-Ardevol

October 30, 2026

Thanks to Alexandre Belling for pointing me in the right direction in the lattice world.

Many relational arguments employed in ZisK, such as the permutation argument of PlonK [GWC19] or the lookup argument of LogUp [Hab22], require the prover $\mathcal{P}$ to commit to a set of polynomials and subsequently demonstrate that these commitments satisfy a system of algebraic constraints at a specific evaluation point, the *challenge*. In the interactive setting, the challenge is sampled uniformly at random and issued by the verifier $\mathcal{V}$. In the non-interactive setting, the standard Fiat–Shamir heuristic is applied: the challenge is derived by hashing the committed polynomials along with additional transcript data required for the soundness of the protocol.

The derivation of challenges becomes substantially more delicate in a *distributed* setting, where the proof of an argument is jointly generated by multiple provers $\mathcal{P}_i$. Specifically, in ZisK, relational arguments are reduced to univariate sumcheck protocols whose evaluation polynomials must be consistently instantiated across all provers. Hence, all provers must operate with an identical challenge value: if provers were to proceed with different evaluation points, the resulting constraints would be incoherent and verification would fail trivially. This necessitates a mechanism to derive a single canonical challenge from the joint set of commitments, and to make it consistently available to all provers before the protocol can proceed.

More precisely, let $\{\mathcal{P}_i\}_{i=1}^{J}$ denote the participating provers. The process unfolds as follows:

1. Each prover $\mathcal{P}_i$ computes a commitment $c_i \in D$ to its local set of polynomials, where D denotes the commitment domain.
2. An accumulator party A (assumed honest, see below) iteratively combines the commitments via an *accumulator function* $\mathrm{acc} : D^k \to D$, which compresses blocks of k commitments into a single value, until all J commitments have been processed.
3. The resulting accumulator value defines the common challenge α, which is broadcast to all provers.

Remark. *For clarity, we omit throughout this note transcript elements such as the public parameters* pp *and auxiliary data, which in practice must also be included to guarantee the soundness of the protocol.*

The accumulator acc is required to satisfy:

- **Efficiency**: acc should be computable in polynomial time, with cost proportional to k.
- **Incrementality**: accumulation should support streaming, i.e., intermediate accumulations should depend only on the commitments observed so far, without recomputing from scratch.
- **Security**: the derived challenge α should be computationally indistinguishable from a uniformly random challenge under standard assumptions, i.e., it should inherit the pseudorandomness guarantees of Fiat–Shamir.

In ZisK, the accumulator is itself embedded inside proof aggregation. Thus, the aggregation constraints guarantee that acc is applied correctly, allowing us to assume that A is honest. We now turn to the cryptographic primitive that enables efficient, incremental and secure accumulation: *multiset hashing*.

1. Multiset Hashing

Multiset hashing, introduced by Clarke et al. [Cla+03] and rooted in the incremental hashing paradigm of Bellare and Micciancio [BM97], provides exactly the required functionality: it supports incremental computation of a hash value over dynamically growing multisets. Multiset hashes are incremental in the sense that, when new members of the multiset become known, the hash can be updated in time proportional to the new members. They must satisfy a stronger notion of security than standard hashes, *multiset collision-resistance*, which says that it must be hard to find two different multisets that produce the same hash.

Formally, a multiset hash function is a map $H : \mathcal{M}(\{0,1\}^*) \to \mathbb{G}$, where $\mathcal{M}(\cdot)$ denotes multisets and $(\mathbb{G}, \oplus)$ is a commutative group, satisfying:

- **Multiset Homomorphism**: for any two multisets $S, T \in \mathcal{M}(\{0,1\}^*)$, $H(S \cup T) = H(S) \oplus H(T)$.
- **Multiset Collision-Resistance**: it is computationally infeasible to find multisets $S, T \in \mathcal{M}(\{0,1\}^*)$ such that $S \neq T$ and $H(S) = H(T)$.

Bellare and Micciancio introduced for sets, and Clarke et al. later generalized to multisets, the idea of taking a collision-resistant hash-to-group function $h : \{0,1\}^* \to \mathbb{G}$ and defining the multiset hash function H over an additive group $(\mathbb{G}, +)$ as

$$H(S) = m_{s_1} h(s_1) + ... + m_{s_J} h(s_J)$$

for the multiset $S = \left\{ s_1^{(m_{s_1})}, ..., s_J^{(m_{s_J})} \right\}$, where $x^{(m_x)}$ denotes that the string x appears m_x times in S, i.e., m_x is the *multiplicity* of x. Multiset homomorphism follows directly from the definition.

Clarke et al. showed that, if $(\mathbb{G}, +)$ is taken to be addition over $\mathbb{Z}_p$, multiset collision-resistance can be proved for a sufficiently large p. This construction, however, requires a secret key, which does not fit our setting. Moreover, Clarke et al. only considered Wagner's attack [Wag02] in their security analysis, and refinements of the attack have appeared since then, e.g., [Sha08], that improve its efficiency.

They also propose to take $(\mathbb{G}, +)$ to be addition over $\mathbb{Z}_p^n$, the space of vectors of dimension n with coefficients in $\mathbb{Z}_p$. In this setting, they show multiset collision-resistance by a reduction to the short integer solution (SIS) problem [Ajt96], a well-studied lattice problem. This latter construction, broadly named LtHash, although less efficient than the previous one, does not require any secret key, and it is the one we focus on from now on.

2. Lattice-Based Multiset Hashing (LtHash)

Fix a positive integer n. Following Clarke et al., we instantiate $(\mathbb{G}, +)$ as the additive group $\mathbb{Z}_p^n$. Define a base hash $h : \{0,1\}^* \to \mathbb{Z}_p^n$, obtained, e.g., by iteratively squeezing a standard cryptographic hash. We define $\text{LtHash} : \mathcal{M}(\{0,1\}^*) \to \mathbb{Z}_p^n$ by

$$\text{LtHash}(S) = \sum_{i=1}^{J} m_{s_i} \cdot h(s_i),$$

where $S = \left\{ s_1^{(m_{s_1})}, ..., s_J^{(m_{s_J})} \right\}$, the sum is taken component-wise over $\mathbb{Z}_p$ and the product is the scalar multiplication of $\mathbb{Z}_p^n$. Again, multiset homomorphism follows directly from the construction. We now turn to multiset collision-resistance.

2.1. Multiset Collision-Resistance.

First, observe that if multiplicities are left unbounded over $\mathbb{Z}_p$, then LtHash is not collision-resistant. Indeed, given a multiset S, take any string $y \notin S$ and form $T = S \cup \{y^{(p)}\}$. Then $T \neq S$ but $\mathrm{LtHash}(T) = \mathrm{LtHash}(S)$. To avoid such trivial collisions, we restrict our attention to $\mathcal{M}^{<B}(\{0,1\}^*)$, the family of multisets of cardinality below $B \ll p$. This is a stronger requirement than asking that every element of a multiset have multiplicity much lower than p, but it turns out to be sufficient for ZisK (see the next section).

Collision-resistance of LtHash can be related to the hardness of the SIS problem, which is believed to remain hard even against quantum adversaries. For sets, an updated reduction is given in Theorem 2.3 of [Lew+19], adapting Theorem 6.1 of [BM97] to modern models; its adaptation to multisets, as in Theorem 3 of [Cla+03], follows directly.

At a high level, the relation goes as follows. For two multisets $S = \left\{ s_1^{(m_{s_1})}, ..., s_J^{(m_{s_J})} \right\}$ and $T = \left\{ t_1^{(m_{t_1})}, ..., t_K^{(m_{t_K})} \right\}$ of $\mathcal{M}^{<B}(\{0,1\}^*)$ with $J, K > 0$, let $U = \mathrm{supp}(S) \cup \mathrm{supp}(T) = \{u_1, ..., u_L\}$, so that $L > 0$, and define

$$A = [h(u_1) \ \cdots \ h(u_L)] \in \mathbb{Z}_p^{n \times L}$$

and $x \in \mathbb{Z}_p^L$ by $x_i = m_{u_i}^S - m_{u_i}^T$, where m_u^X denotes the multiplicity of u in the multiset X. Then finding a multiset collision amounts to producing two multisets $S \neq T$ such that

$$\sum_{i=1}^{J} m_{s_i} \cdot h(s_i) - \sum_{i=1}^{K} m_{t_i} \cdot h(t_i) = \mathbf{0} \quad \Longleftrightarrow \quad Ax = \mathbf{0}.$$

Since both S and T have bounded cardinality and h is a secure hash function, a collision yields a nontrivial solution x of the SIS instance with infinity norm below B. Consequently, under the hardness of SIS, it is infeasible to find collisions of LtHash.

2.2. **Enforcing Bounded Cardinality.**

As explained before, the size of every multiset must remain bounded. A simple and effective technique is to augment each vector with an additional coordinate fixed to 1, so that every accumulation tracks the total multiplicity of its elements, providing a built-in mechanism to enforce the cardinality bound.

Formally, define the augmented embedding $h'(x) = (h(x), 1) \in \mathbb{Z}_p^{n+1}$. The LtHash of a multiset $S = \left\{ s_1^{(m_{s_1})}, ..., s_J^{(m_{s_J})} \right\}$ is then

$$\mathrm{LtHash}(S) = \sum_{i=1}^{J} m_{s_i} \cdot h'(s_i) = \left(\sum_{i=1}^{J} m_{s_i} \cdot h(s_i), \sum_{i=1}^{J} m_{s_i} \right).$$

The last coordinate records the total multiplicity of the multiset. By requiring, within proof aggregation, that this coordinate stays strictly below a fixed bound B (e.g., 2^{20}), we guarantee that no multiset reaches B elements. This prevents adversaries from building long addition chains or otherwise bypassing the cardinality bound.

2.3. **Concrete Parameters and Security.**

We use the Lattice Estimator [Alb24] to estimate the security of LtHash against the best known attacks on SIS. With $n = 372$, $p = 2^{64} - 2^{32} + 1$ and $B = 2^{20}$,

```sage
from estimator import *

vector_size = 372
goldilocks = 2**64 - 2**32 + 1
bound = 2**20

params = SIS.Parameters(n=vector_size, q=goldilocks, length_bound=bound, norm=oo)
SIS.estimate(params)
```

we obtain

```
lattice  :: rop: ≈2^134.3, red: ≈2^133.3, sieve: ≈2^133.3,
            β: 363, η: 373, ζ: 317, d: 1951, prob: 1, ↻: 1, tag: infinity
```

which is sufficient for our 128-bit security target.

3. Application to Distributed Challenge Derivation in ZisK

We now describe how distributed challenge derivation is instantiated in ZisK. Let n be a positive integer, let $p = 2^{64} - 2^{32} + 1$ be the Goldilocks prime, and let h be the Poseidon2 hash function [GKS23]. ZisK builds on STARKs as its underlying proving system, where polynomial commitments are the Merkle roots of trees containing all polynomial evaluations over a predefined domain over $\mathbb{Z}_p$. Since Poseidon2 is used in the commitment phase, every commitment c_i lies in the domain $D = \mathbb{Z}_p^4$.

For security, the challenge space must be large enough for the target security level. As ZisK targets 128-bit security, we fix $\mathbb{Z}_p^3$ as the challenge space. Finally, we use the technique of the previous section to bound the number of accumulated elements by B.

The distributed challenge α is computed collaboratively by the provers $\{\mathcal{P}_i\}_{i=1}^{J}$ as follows:

1. **Commit phase.** Each prover $\mathcal{P}_i$ computes its polynomial commitment $c_i \in \mathbb{Z}_p^4$ and sends it to the designated accumulator party A.
2. **Initialization and first accumulation.** The accumulator is initialized at $\text{acc}_0 = 0$. Upon receiving the first batch of k commitments $C = \{c_1, ..., c_k\}$, the accumulator builds the multiset $S = \left\{ s_1^{(m_{s_1})}, ..., s_\ell^{(m_{s_\ell})} \right\}$ from C. For each $s_i \in S$, it derives a vector embedding by iterated hashing,
$$h_{i,1} = h(s_i), \quad h_{i,2} = h\big(h_{i,1}\big), \quad ..., \quad h_{i,n/4} = h\Big(h_{i,n/4-1}\Big),$$
where each $h_{i,j} \in \mathbb{Z}_p^4$, and concatenates all intermediate values into
$$v_i = \Big(h_{i,1}, ..., h_{i,n/4}, 1\Big) \in \mathbb{Z}_p^{n+1}.$$
It then applies LtHash:
$$\text{acc}_1 = \text{acc}_0 + \text{acc}(c_1, ..., c_k) = \text{LtHash}(S) = \sum_{i=1}^{\ell} m_{s_i} \cdot v_i.$$
3. **Subsequent accumulations.** As further batches of k commitments arrive, the accumulator repeats the above process, updating $\text{acc}_i = \text{acc}_{i-1} + \text{acc}(\cdot)$.
4. **Finalization.** After processing all J commitments, i.e., after J/k accumulation rounds, the accumulator outputs the final value $\text{acc} := \text{acc}_{J/k}$, which is broadcast to all the provers. Each prover continues the protocol using the embedding of acc into $\mathbb{Z}_p^3$ as the common challenge α.

Efficiency, incrementality and security follow directly from the properties of LtHash. Accumulation is fully parallelizable and can be performed in any order. Input lengths can also vary, $k' \neq k$ being permissible, so several accumulators can be composed hierarchically: two accumulators A_1 and A_2 may independently process disjoint subsets C_1 and C_2 of the commitments $C = \{c_1, ..., c_J\}$, after which a final aggregation step combines their outputs.

References

[Ajt96] Miklós Ajtai. Generating Hard Instances of Lattice Problems (Extended Abstract). In *28th Annual ACM Symposium on Theory of Computing*, pages 99–108. ACM Press, May 1996. doi:10.1145/237814.237838.

[Alb24] Martin R. Albrecht. Lattice Estimator. GitHub repository, 2024. URL: https://github.com/malb/lattice-estimator.

[BM97] Mihir Bellare and Daniele Micciancio. A New Paradigm for Collision-Free Hashing: Incrementality at Reduced Cost. In Walter Fumy, editor, *Advances in Cryptology – EUROCRYPT'97*, volume 1233 *of Lecture Notes in Computer Science*, pages 163–192. Springer, Berlin, Heidelberg, May 1997. doi:10.1007/3-540-69053-0_13.

[Cla+03] Dwaine E. Clarke, Srinivas Devadas, Marten van Dijk, Blaise Gassend, and G. Edward Suh. Incremental Multiset Hash Functions and Their Application to Memory Integrity Checking. In Chi-Sung Laih, editor, *Advances in Cryptology – ASIACRYPT 2003*, volume 2894 *of Lecture Notes in Computer Science*, pages 188–207. Springer, Berlin, Heidelberg, November 2003. doi:10.1007/978-3-540-40061-5_12.

[GKS23] Lorenzo Grassi, Dmitry Khovratovich, and Markus Schofnegger. Poseidon2: A Faster Version of the Poseidon Hash Function. In Nadia El Mrabet, Luca De Feo, and Sylvain Duquesne, editors, *AFRICACRYPT 23: 14th International Conference on Cryptology in Africa*, volume 14064 *of Lecture Notes in Computer Science*, pages 177–203. Springer, Cham, July 2023. doi:10.1007/978-3-031-37679-5_8.

[GWC19] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge. Cryptology ePrint Archive, Report 2019/953, 2019. URL: https://eprint.iacr.org/2019/953.

[Hab22] Ulrich Haböck. Multivariate lookups based on logarithmic derivatives. Cryptology ePrint Archive, Report 2022/1530, 2022. URL: https://eprint.iacr.org/2022/1530.

[Lew+19] Kevin Lewi, Wonho Kim, Ilya Maykov, and Stephen Weis. Securing Update Propagation with Homomorphic Hashing. Cryptology ePrint Archive, Report 2019/227, 2019. URL: https://eprint.iacr.org/2019/227.

[Sha08] Andrew Shallue. An Improved Multi-set Algorithm for the Dense Subset Sum Problem. In *Algorithmic Number Theory, 8th International Symposium, ANTS-VIII*, volume 5011 *of Lecture Notes in Computer Science*, pages 416–429. Springer, Berlin, Heidelberg, 2008. doi:10.1007/978-3-540-79456-1_28.

[Wag02] David Wagner. A Generalized Birthday Problem. In Moti Yung, editor, *Advances in Cryptology – CRYPTO 2002*, volume 2442 *of Lecture Notes in Computer Science*, pages 288–303. Springer, Berlin, Heidelberg, August 2002. doi:10.1007/3-540-45708-9_19.